Tipos Abstratos de Dados

Professor: André Felipe Ribeiro Cordeiro
Disciplina: Estruturas de Dados

Departamento de Informática
Centro de Tecnologia
Universidade Estadual de Maringá

Introdução

- Algoritmos fazem parte do dia a dia das pessoas. **Instruções** para o uso de **medicamentos** e indicações de como **montar um aparelho** são exemplos de algoritmos.

- Um algoritmo pode ser visto como uma **sequência de ações**, para obter uma **solução** relacionada com um **problema**.

Introdução

- Além do estudo de **algoritmos**, o estudo das **estruturas de dados** também é importante.

- Estruturas de dados e algoritmos estão **relacionados**.

Introdução

- Não se pode estudar estruturas de dados **sem considerar** os algoritmos associados a elas.

- A **escolha dos algoritmos** depende da **representação** e da **estrutura** dos dados.

Introdução

- Considerando a interação entre algoritmos e estruturas de dados, pode-se dizer que **programar** é basicamente **estruturar dados** e **desenvolver algoritmos**.

Programa = algoritmo + estruturas de dados

- ✓ algoritmo : **lógica** do programa
- ✓ estruturas de dados: **dados** do programa

Estruturas de Dados

- Estruturas de Dados são **formas genéricas** de se estruturar dados, de modo a **facilitar o registro** e **processamento** pelo computador.

- **Exemplos**
  - Listas, Filas, Pilhas e Árvores

Tipos de Dados

- Os algoritmos e as estruturas manipulam **dados de determinados tipos**.

- Em linguagens de programação, é possível **classificar** constantes, variáveis, expressões e funções, a partir do **tipo de dado** das mesmas.

Tipos de Dados

- **Tipos simples** de dados são grupos de valores indivisíveis, que podem assumir valores em um determinado intervalo.

- **Exemplos**
  - int, char, float, double..

Tipos de Dados

- Em determinadas situações, a utilização de tipos simples é **suficiente** para resolver um problema.

- Em outras situações, precisa-se utilizar agregados, tais como **vetores** ou **matrizes**.

Tipos de Dados

- **Vetores**
  - arranjo unidimensional de dados de um tipo.

- **Matrizes**
  - arranjo de várias dimensões de dados de um tipo.

Observação

- Vetores e matrizes são **agregados homogêneos**.

Vector **A**

Conteúdo	12	9	10	16	25	13	20	14		14
Posição	0	1	2	3	4	5	6	7	8	9

Matriz **B**

	0	1	2	3	4
0	A	E	I	O	U
1	X	A	Y	I	O
2	G	A	S	J	A
3	A	F	D		A
4	T	X	C	Q	E

Tipos de Dados

- Quando vetores e/ou matrizes não são suficientes, pode-se ainda utilizar **agregados heterogêneos**.

```
struct mystruct
{
   char  a;
   int   b;
   float c;
};

struct mystruct myvar;

myvar.b = 99;
```

Tipo
Abstrato de Dados

- Os **tipos estruturados** definem uma **coleção** de valores simples ou **agregado** de valores de tipos diferentes.

- Um Tipo Abstrato de Dados (TAD) pode ser visto como um **tipo estruturado**, acrescido de um **conjunto de operações**.

Tipo
Abstrato de Dados

- Em linguagem C, vamos utilizar **estruturas** e **operações** para implementar TADs.

- As operações são definidas por meio de **funções**.

Exemplo

- **TAD Fração**
  - Estrutura
    - ✓ campos numerador e denominador
  - Operações
    - ✓ adicionar e multiplicar

Tipo
Abstrato de Dados

Estrutura

```
typedef struct {
  int numerador;
  int denominador;
}Fracao;
```

Operação

```
void multiplicar(Fracao*, Fracao*, Fracao*);
```

Tipo
Abstrato de Dados

Implementação

```
void multiplicar
      (Fracao* f1, Fracao* f2, Fracao* f){
  f->numerador =
    f1->numerador * f2->numerador;
  f->denominador =
    f1->denominador * f2->denominador;
}
```

Observações

- A definição da **estrutura** e das **operações** podem estar localizadas em uma única seção do programa. **Diferentes TADs podem ser definidos**.

- **Cada conjunto de operações estabelece um TAD diferente**, mesmo que todos os conjuntos de operações considerem a **mesma estrutura**.

Observações

- A representação do TAD não se relaciona a princípio com **nenhum detalhe de implementação**.

- A especificação de um TAD favorece atividades como **alteração** e **correção de erros**, além de **ocultar detalhes de implementação**.

Observações

- TADs são utilizados extensivamente em **projetos de algoritmos**.

- A Linguagem de Programação (LP) utilizada deve fornecer uma forma de **representar o TAD**, em termos de **estrutura** e **operações**.

Referências

- Projeto de Algoritmos com implementações em Pascal e C, Nivio Ziviani, 2011.

- Estruturas de Dados usando C, Aaron Tenenbaum, Yedidyah Langsam e Moshe Augenstein, 2013.

Referências

- Introdução a Estruturas de Dados, Waldemar Celes, Renato Cerqueira e José Rangel, 2016.

- Aulas Professor Rodolfo Miranda Pereira, UEM.